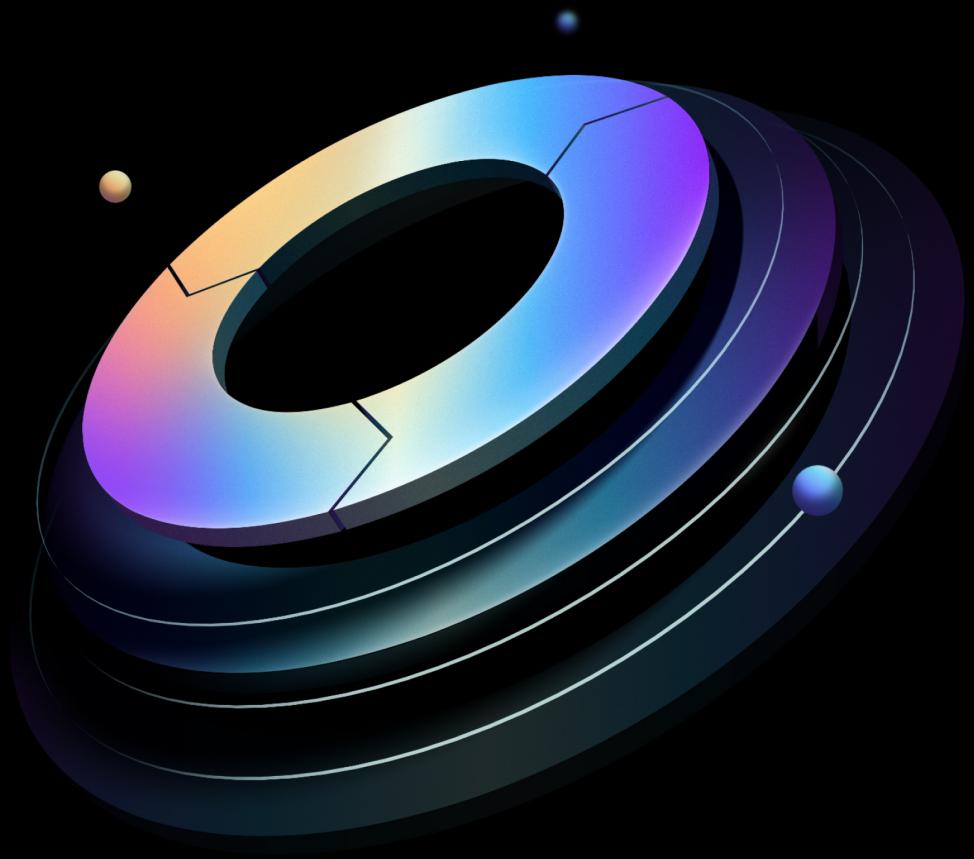
HashiCorp

Vault Radar Administration Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **2**
1.1 Why use HashiCorp Validated Designs? . . . 2
1.2 What this guide covers . . . 3
1.3 Related guides . . . 3

2 People and process **3**

3 Identity and access management **4**
3.1 Resource-based access control . . . 4

4 Data source onboarding **5**
4.1 Connect data sources . . . 6
4.2 Tune secrets detection . . . 6
4.2.1 Reduce false positive fatigue . . . 6

5 Agent deployment **7**
5.1 Prerequisites . . . 7
5.2 Agent horizontal scaling . . . 11

6 Observability and alert management **13**
6.1 Configure alerting . . . 13
6.2 Configure ticketing . . . 13
6.3 Notification integrations . . . 13

7 Changelog **14**
7.1 2026-08 . . . 14

8 Credits **14**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

Note

This guidance for HCP Vault Radar is *emerging guidance*. We continually update it as we identify common best practices and configurations to help customers get the most from the product.Throughout this guide, the terms HCP Vault Radar, Vault Radar, and Radar are interchangeable. There is no self-managed or enterprise version of Vault Radar.

This Administration Guide covers the ongoing configuration, operation, and governance of HCP Vault Radar for the **platform, security operations (SecOps), and HCP administrator teams** who run Radar as a service for their organization.

Because Vault Radar is a software-as-a-service (SaaS) offering delivered through the HashiCorp Cloud Platform (HCP), there is no self-managed installation to perform: HashiCorp operates the underlying platform. This guide therefore assumes your organization already has access to an HCP

organization with Vault Radar enabled. Infrastructure concerns that do not apply to a fully-managed SaaS product — such as backup, disaster recovery, upgrades, and capacity sizing — are handled by HashiCorp and are out of scope for this guide.

1.2 What this guide covers

- **People and process** — the roles and responsibilities involved in running Vault Radar.
- **Identity and access management** — HCP roles and resource-based access control for operators and consumers.
- **Data source onboarding** — connecting and tuning the data sources Radar scans.
- **Agent deployment** — deploying and scaling agents for scanning private, non-internet-accessible sources.
- **Observability and alert management** — configuring alerting, notification, and ticketing integrations.

1.3 Related guides

For how developers and application teams *consume* Vault Radar — deployment models, using Radar in developer pipelines, and remediating findings — see the User Guide.

2 People and process

Implementing Vault Radar requires clear definition of roles and responsibilities across your organization. Different teams need varying levels of access and permissions to effectively manage secret detection and remediation workflows. The following outlines the key stakeholders involved in Vault Radar operations and their recommended access levels:

- **Developer and operations (DevOps) teams** receive alerts when Vault Radar identifies a secret in a data source. They are typically granted the *viewer* role, which allows them to view events, or the *contributor* role, which allows them to update remediation states.
- **Security operations (SecOps) teams** onboard data sources, set up alerts, understand secret sprawl, generate reports, and assign remediation tasks. SecOps teams also own the task of identifying and managing the remediation of secret sprawl at the company level. To perform these tasks, assign each member the *admin* role. The admin role grants broad read/write

capabilities and provides the ability to onboard new users, delete projects, and configure platform level settings.

- **Chief information security officer (CISO)** needs insights into how effective various teams are at eliminating hard coded secrets in code and moving them to a secrets manager like HashiCorp Vault. Assign this team the *viewer* role, to only see the dashboard and information.
- **Platform teams/HCP administrators** configure and administer the tool configuration, including integrations (data sources, alerting, ticketing), agents and remediations. Although this role overlaps with the SecOps team, it works in collaboration with them. Assign this team the *admin* role.

For the mechanics of assigning these HCP roles and configuring resource-based access control, see Identity and access management.

3 Identity and access management

Vault Radar access is governed through HashiCorp Cloud Platform (HCP) identity and access management. Operators assign organization-level roles to teams — see People and process for the recommended role per stakeholder — and then use resource-based access control to grant least-privileged access to specific data sources.

3.1 Resource-based access control

Vault Radar administrators can grant developer access to specific resources for least-privileged access at the group level.



Figure 1: Resource-based access control configuration

Create groups at the HCP organization level, and then assign them to one or more data sources or repositories with Resource Contributor or Viewer roles. While the Resource Viewer role can

only view information about a risk, a Resource Contributor can view and modify the state of a risk, possibly applying remediation to it.

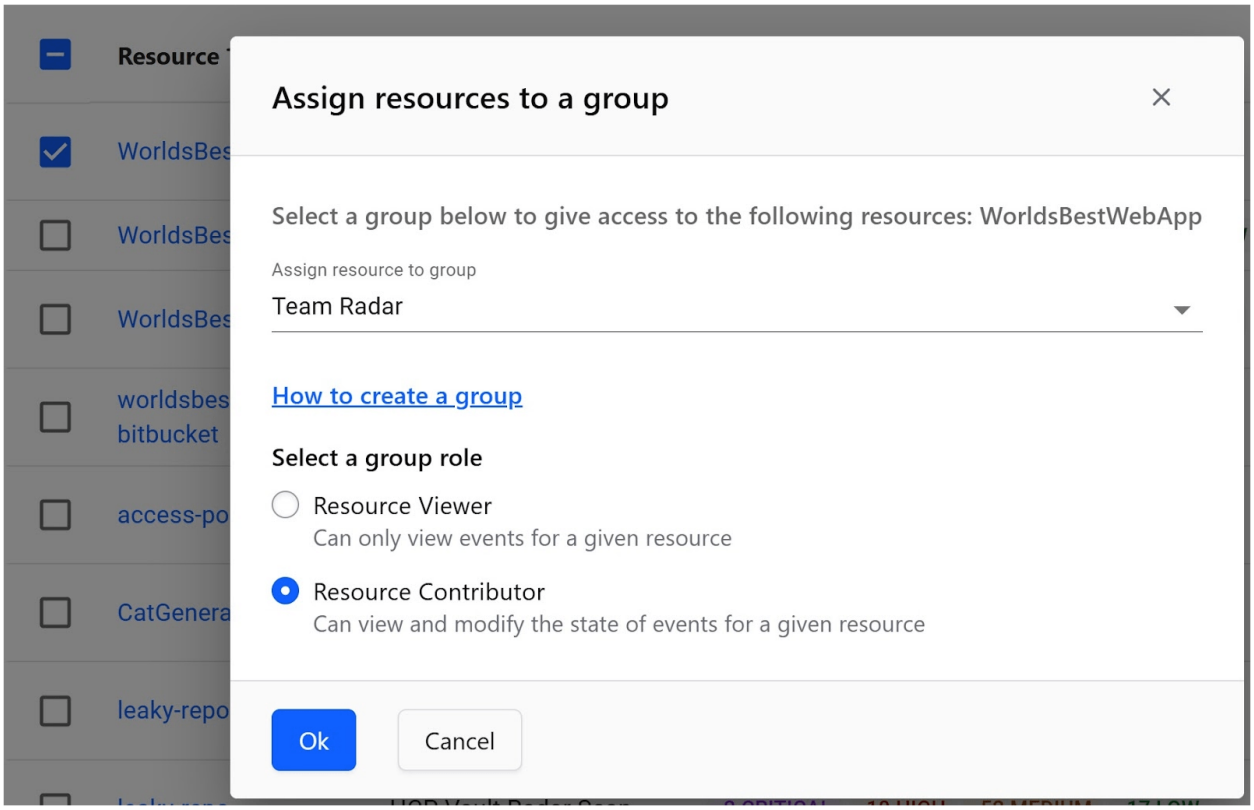


Figure 2: Group and role assignment interface

For more on HCP roles and access management, see the HCP access management documentation.

4 Data source onboarding

Onboarding data sources is an operator task: the platform or SecOps team connects the repositories and collaboration tools that Vault Radar scans, then tunes detection so that findings are accurate and actionable.

> **Note**
>
> The Terraform provider for Vault Radar is in private beta.

4.1 Connect data sources

Follow the documentation page for connecting a data source for the following supported platforms:

- GitHub
- GitLab
- Confluence Cloud and Confluence Server
- BitBucket
- Azure DevOps

As mentioned within the Vault Radar documentation, there is a limit of 20,000 repositories per data source. If you have more than 5,000 repositories, Vault Radar selects the 5,000 repositories with the most recent activity. To onboard more repositories, you need to create more connections and select the remaining repositories.

To scan private, non-internet-accessible sources, onboard them through a Vault Radar agent — see Agent deployment.

4.2 Tune secrets detection

After connecting a data source, tune the secrets detection with regex patterns and rules:

- Custom Regex
- Global Ignore Rules
- Local Ignore Rules
- Event Rules

4.2.1 Reduce false positive fatigue

To combat false positive fatigue, security teams should focus on improving detection engineering, tuning Vault Radar, and enhancing alert context. This involves reducing the volume of irrelevant

alerts, providing more information about alerts, and automating investigations to prioritize genuine threats.

Key strategies to reduce false positive fatigue are:

- **Prioritize by severity level**: Focus on alerts with higher severity and potential impact, rather than overwhelming analysts with a large volume of low-priority alerts.
- **Test custom regex patterns**: Thoroughly test any new or modified detection rules to ensure they function as expected and don't introduce unintended consequences.
- **Monitor false positives**: Regularly monitor and analyze false positives to identify patterns and adjust rules accordingly, improving the overall accuracy of the scanning process.
- **Exclude directories with mock data**: This helps prevent Vault Radar from flagging mock data that is intentionally not sensitive.
- **Manage ignore rules**: The same applies for developing Global/repository specific ignore rules to reduce the flagging of findings that are not actually sensitive.

5 Agent deployment

Deploying agents is an operator task for hybrid scanning of resources that are not publicly accessible by HCP. This page covers the prerequisites, resource requirements, data source connection, and horizontal scaling of Vault Radar agents.

5.1 Prerequisites

- Outbound connectivity to the HCP platform
- An agent pool in the HCP Portal
- An HCP project level service principal with the Vault Radar agent role assigned
- Base 64 encoded data source credential (e.g. GitHub/GitLab personal access token (PAT), Bitbucket HTTP access token, etc.)

The agent supports scanning within a private network. It connects with HCP for licensing and uploading only metadata to provide centralized secret risk inventory. It does not send secret values or sensitive information to HCP.

For deploying the agent, follow the Deploy Vault Radar Agent documentation page. Kubernetes is currently our recommended platform for deploying the Agent. You are welcome to extrapolate the

configuration for other platforms, but we cannot guarantee support for that platform at this time.

For recommended resource requests and limits, refer to the comments in the Kubernetes manifest sample.

Note that in our Kubernetes manifest sample, we have set up a secret and environment variable for connecting to our data sources, in this case a GitHub PAT. You can set up any supported code server token like the GitHub PAT example below, by creating a Kubernetes Secret resource, and injecting it as a Kubernetes deployment environment variable.

> **Note**
>
> You can use a different variable name. But you must be consistent with the name when configuring the data source below.

If you are using a Linux server, set an environment variable manually at the target machine:

```
export VAULT_RADAR_GIT_TOKEN=<YOUR_GITHUB_PAT>
```

After the agent connects and is running, add a data source for scanning. At the Vault Radar portal page, click on Settings > Data Sources > Add Data Source. Select Vault Radar Agent Scan:

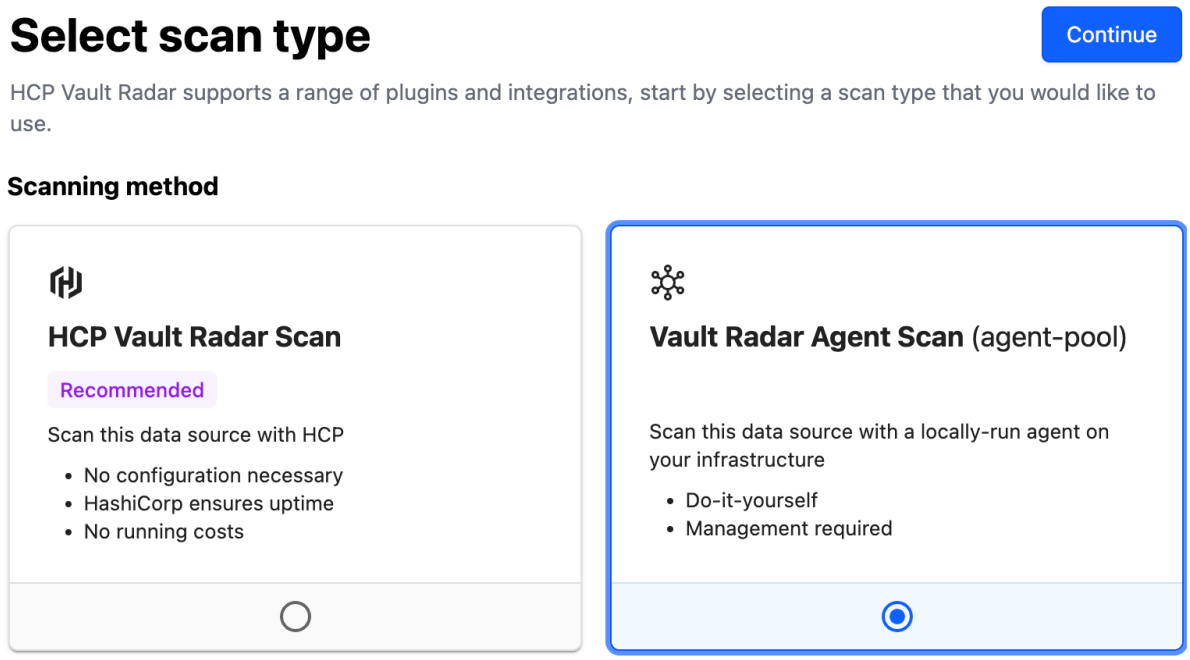


Figure 3: Select Vault Radar Agent Scan option

Select your **Code Server** and fill the connection information. Enter the environment path for your code server access token in the last field:

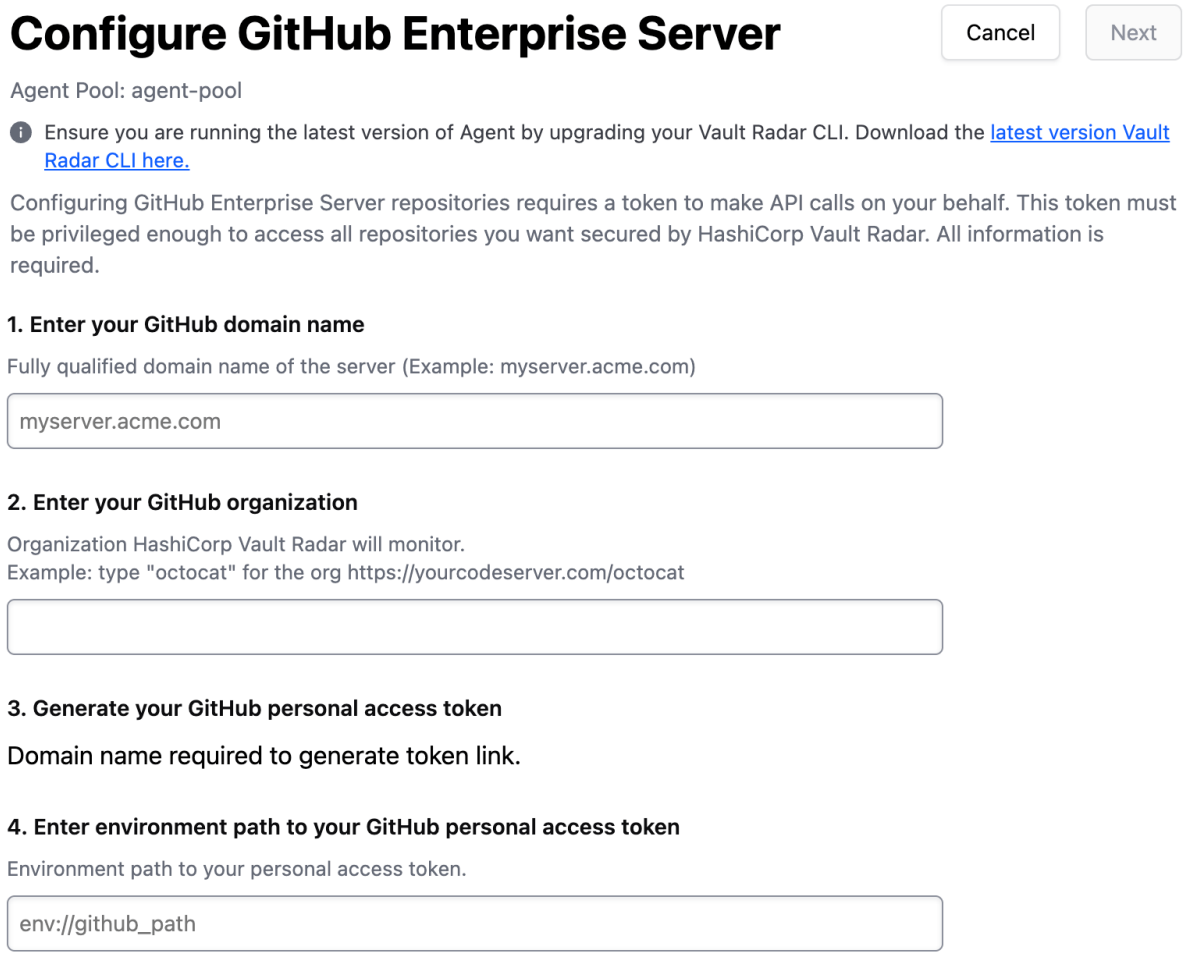


Figure 4: Configure code server connection settings

Check that the URI entered is correct. It should be `env://<YOUR_VARIABLE_NAME>`. In the example this is: `env://VAULT_RADAR_GIT_TOKEN`

Click next and wait for the agent to validate the configuration. On the next page, **select repositories to monitor** to set up the data source. When ready you can select finish.

For additional information, please check the Deploy Vault Radar agent page.

5.2 Agent horizontal scaling

Scaling the agent is more complex due to its nature. It handles a variety of operations and is in theory, always running. The agent is **horizontally scaled** and there are some configuration options available to optimize the agent pool.

By default when the Agent starts, it runs and polls for all types of work. The works best when the aim is scanning happen in the background. When the operation is one with a user waiting for the results, it can result in users waiting for results behind other workloads.

One way to address this is to deploy more agents using the same Agent Pool ID to scale the agent. A better approach is to refine the agent deployment to only run a specific workload. This allows users to scale multiple agent deployments to handle background scanning and user facing workloads like PR Scanning.

Workload	Purpose	Interaction Type	Command (Default to `true`)
Full resource scanning	This is a full scan of a resource. This is typically something that happens as a background task, but users can request an ad-hoc full scan.	Typically background	`AGENT_POLL_RESOURCE_SCAN_JOB_E`
Webhook content scanning	This is a scan of some change to a resource (like a commit to a repository). This is generally a background operation.	Typically background	`AGENT_POLL_CONTENT_SCAN_JOB_EN`

Workload	Purpose	Interaction Type	Command (Default to `true`)
Data source onboarding	These workloads are requests made as part of onboarding a new data source. Often a user is expecting this to complete quickly.	Typically user facing	`AGENT_POLL_HTTP_JOB_ENABLED`
Pull request scanning	This workload is responsible for scanning a PR and annotating the contents. This is a user facing operation.	Typically user facing	`AGENT_POLL_PULL_REQUEST_SCAN_J`
Copy secrets	This workload is responsible for handling Copy Secrets. This is also typically a user facing operation.	Typically user facing	`AGENT_POLL_COPY_SECRETS_JOB_EN`

An example of how to use this by creating two deployments is below. Both deployments share the same Agent Pool ID, Service Principals, and any Credentials. The first deployment is for background operations only. To do that disable the user facing operations flags by setting:

```
AGENT_POLL_COPY_SECRETS_JOB_ENABLED=false
AGENT_POLL_PULL_REQUEST_SCAN_JOB_ENABLED=false
AGENT_POLL_HTTP_JOB_ENABLED=false
```

And in the other deployment we can disable the background workload flags by setting:

```
AGENT_POLL_CONTENT_SCAN_JOB_ENABLED=false
AGENT_POLL_RESOURCE_SCAN_JOB_ENABLED=false
```

Scale these two deployments separately depending on the need for background or user facing

scanning.

6 Observability and alert management

Operators configure how Vault Radar findings are surfaced and routed. Vault Radar integrates with ticketing and alerting systems to provide real-time alerting and automation. Alerting and automation enable faster time to resolution and fit directly into existing security and engineering workflows.

6.1 Configure alerting

Follow the documentation page for configuring alerting for the following supported platforms:

- PagerDuty
- Slack and Slack App
- Splunk
- Microsoft Teams

6.2 Configure ticketing

Follow the documentation page for configuring ticketing for the following supported platforms:

- Jira
- ServiceNow

6.3 Notification integrations

Notification integrations allow Vault Radar administrators to notify development teams when there is an event that needs remediation. We support integrations with:

- Microsoft Teams
- Jira
- ServiceNow
- Splunk

- PagerDuty
- Slack

Once notifications reach a development team, the consumer-facing remediation experience is covered in the User Guide: Remediation workflows.

7 Changelog

7.1 2026-08

- Restructured legacy Vault Radar HVD content into the HVD 2.0 Vault Radar Administration Guide as part of the Installation / Administration / User guide migration.
- As a fully-managed SaaS product, Vault Radar has no Installation Guide.

8 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Andre Pimentel
- Cami Cano Gomez
- David Mills
- Mark Lewis